

old_01_pipeline

September 21, 2025

```
[1]: import pandas as pd
import numpy as np
import joblib, warnings, os
warnings.filterwarnings("ignore")
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import mean_absolute_error, root_mean_squared_error, r2_score

from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
import xgboost as xgb
import lightgbm as lgb

# -----
# 1. Chargement
# -----
FILE = "regularite-mensuelle-tgv-aqst.csv"
df = pd.read_csv(FILE, sep=";", encoding="utf-8")

# miniscules, retire les espaces
df.columns = df.columns.str.strip().str.lower().str.replace(" ", "_")

# Colonne Date
df["periode"] = pd.to_datetime(df["date"], format="%Y-%m")
df = df.sort_values("periode") # ordre chronologique

# -----
# 2. FEATURE ENGINEERING
# -----
# calendrier
df["mois"] = df["periode"].dt.month
df["jour_semaine"] = df["periode"].dt.dayofweek

# régularité et retard
df["taux_regularite"] = 100 - df["nombre_de_trains_en_retard_au_départ"] / df["nombre_de_circulations_prévues"] * 100
```

```

df["taux_retard"]      = 100 - df["taux_regularite"]

# lags / moyenne mobile
df["lag_1"] = df["taux_regularite"].shift(1)
df["lag_7"] = df["taux_regularite"].shift(7)
df["ma_7"]  = df["taux_regularite"].rolling(7).mean()
drop_list = ['commentaire_annulations', 'commentaire_retards_au_départ',
             ↪ "commentaire_retards_à_l'arrivée"]
df.drop(drop_list, axis=1, inplace=True)
df = df.dropna()

# -----
# 3.  TRAIN / TEST
# -----
FEATURES = ["mois", "jour_semaine", "nombre_de_circulations_prévues",
            "lag_1", "lag_7", "ma_7"]
TARGET   = "taux_regularite"

X = df[FEATURES]
y = df[TARGET]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, shuffle=False)

# -----
# 4.  Modèles + CV temporelle + sélection + fit final
# -----
from sklearn.model_selection import TimeSeriesSplit

models = {
    "Linear Regression": LinearRegression(),
    "Random Forest": RandomForestRegressor(n_estimators=300, random_state=1,
    ↪ n_jobs=-1),
    "XGBoost": xgb.XGBRegressor(n_estimators=300, learning_rate=0.05,
    ↪ max_depth=4, objective='reg:squarederror'),
    "LightGBM": lgb.LGBMRegressor(n_estimators=300, learning_rate=0.05,
    ↪ max_depth=4, objective='regression_l2', verbose=-1)
}

results = {}

# -----
# 4b. Cross-validation temporelle (MAE + RMSE + R²)
# -----
from sklearn.model_selection import cross_validate
from sklearn.metrics import make_scorer

```

```

# scorers supplémentaires
scorers = {
    'mae' : make_scorer(mean_absolute_error, greater_is_better=False),
    'rmse' : make_scorer(root_mean_squared_error, greater_is_better=False),
    'r2' : make_scorer(r2_score, greater_is_better=True)
}

tscv = TimeSeriesSplit(n_splits=5)

for name, model in models.items():
    pipe = Pipeline([('scaler', StandardScaler()), ('reg', model)])

    cv_scores = cross_validate(pipe, X_train, y_train,
                               scoring=scorers,
                               cv=tscv, n_jobs=-1, return_train_score=False)

    results[name] = {
        "CV_MAE" : -cv_scores['test_mae'].mean(),
        "CV_RMSE" : -cv_scores['test_rmse'].mean(),
        "CV_R2" : cv_scores['test_r2'].mean(),
        "CV_STD" : cv_scores['test_mae'].std()
    }

    print(f"{name:15s} - "
          f"MAE : {results[name]['CV_MAE']:.2f} % "
          f"RMSE : {results[name]['CV_RMSE']:.2f} % "
          f"R² : {results[name]['CV_R2']:.3f}")

# -----
# 5. Choix + fit final
# -----
best_name = min(results, key=lambda k: results[k]["CV_MAE"])
best_model = models[best_name]
print(f"\nMeilleur modèle (CV) : {best_model.__class__.__name__} - MAE : ␣
↳ {results[best_name]['CV_MAE']:.2f} %")

final_pipe = Pipeline([('scaler', StandardScaler()), ('reg', best_model)])
final_pipe.fit(X_train, y_train) # entraînement sur 100 % train

# -----
# 6. Évaluation (hold-out)
# -----
y_holdout = final_pipe.predict(X_test)
hold_mae = mean_absolute_error(y_test, y_holdout)
print(f"MAE hold-out final : {hold_mae:.2f} %")

# -----

```

```

# 7. Sauvegarde
# -----
MODEL_PATH = "best_punctuality_model.pkl"
joblib.dump(final_pipe, MODEL_PATH)
print(" best_punctuality_model.pkl sauvegardé")
print(f"Modèle sauvegardé sous {MODEL_PATH}")

# -----
# 8. Interprétation
# -----
trained_model = final_pipe.named_steps["reg"]
feat_names = FEATURES

if isinstance(trained_model, LinearRegression):
    # impact moyen en points de régularité
    coef_df = pd.DataFrame({
        "feature": feat_names,
        "coef" : trained_model.coef_,
        "impact" : trained_model.coef_ * X_train.std() # pts pour 1
    }).sort_values("impact", key=abs, ascending=False)
    joblib.dump(coef_df, "linear_impact.bz2")

# -----
# 9. Analyses métier
# -----
class Analyzer:
    def __init__(self, sncf_df=None):
        self.sncf_df = sncf_df

    def analyze_sncf_performance(self):
        if self.sncf_df is None:
            return {}
        res = {}
        # meilleure période
        best = self.sncf_df.loc[self.sncf_df["taux_regularity"].idxmax()]
        res["best_regularity_period"] = {
            "period": best["periode"].strftime("%Y-%m"),
            "rate" : best["taux_regularity"],
            "service": best.get("service", "N/A"),
            "liaison": f"{best['gare_de_départ']} → {best['gare_d'arrivée"]}",
        }
        # liaisons problématiques
        self.sncf_df["taux_retard"] = 100 - self.sncf_df["taux_regularity"]
        worst = (
            self.sncf_df.groupby(["gare_de_départ", "gare_d'arrivée"])
            .agg(
                taux_retard=("taux_retard", "mean"),

```

```

        nombre_de_trains=("nombre_de_circulations_prévues", "sum"),
    )
    .sort_values("taux_retard", ascending=False)
    .head(10)
    .reset_index()
)
worst["liaisons"] = worst["gare_de_départ"] + " → " +
↳worst["gare_d'arrivée"]
res["problematic_liaisons"] = worst[["liaisons", "taux_retard",
↳"nombre_de_trains"]]

# tendances mensuelles
monthly = (
    self.sncf_df.groupby("mois")
    .agg(
        taux_regularity=("taux_regularity", "mean"),
        nombre_de_trains=("nombre_de_circulations_prévues", "sum"),
    )
    .reset_index()
)
res["monthly_trends"] = monthly
return res

# instancier et remplir le dict
analyser = Analyzer(sncf_df=df)
analyses = {
    "sncf_performance": analyzer.analyse_sncf_performance()
}

# -----
# 10. Sauvegardes finales
# -----
joblib.dump(analyses, "analyses.bz2")
print(" analyses.bz2 sauvegardé")

```

Linear Regression - MAE : 14.70 % RMSE : 18.37 % R^2 : 0.263
 Random Forest - MAE : 15.41 % RMSE : 19.26 % R^2 : 0.204
 XGBoost - MAE : 15.25 % RMSE : 19.10 % R^2 : 0.220

```

/Users/markus/miniconda3/lib/python3.12/site-
packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid
feature names, but LGBMRegressor was fitted with feature names
    warnings.warn(
/Users/markus/miniconda3/lib/python3.12/site-
packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid
feature names, but LGBMRegressor was fitted with feature names
    warnings.warn(

```

```

/Users/markus/miniconda3/lib/python3.12/site-
packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid
feature names, but LGBMRegressor was fitted with feature names
  warnings.warn(
/Users/markus/miniconda3/lib/python3.12/site-
packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid
feature names, but LGBMRegressor was fitted with feature names
  warnings.warn(

LightGBM      - MAE : 15.07 %  RMSE : 18.80 %  R2 : 0.241

Meilleur modèle (CV) : LinearRegression - MAE : 14.70 %
MAE hold-out final : 14.25 %
  best_punctuality_model.pkl sauvegardé
Modèle sauvegardé sous best_punctuality_model.pkl
  analyses.bz2 sauvegardé

/Users/markus/miniconda3/lib/python3.12/site-
packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid
feature names, but LGBMRegressor was fitted with feature names
  warnings.warn(

```

[]: